



DALHOUSIE UNIVERSITY

CSCI 5408: Data Management, Warehousing and Analytics

Assignment: 02

Banner ID: B01026328

Name: Parva Mineshbhai Patel

Date: 2025 – 04 – 06

GitLab Assignment Link: [https://git.cs.dal.ca/parva/
CSCI5408_W25_B01026328_ParvaMineshbhai_Patel](https://git.cs.dal.ca/parva/CSCI5408_W25_B01026328_ParvaMineshbhai_Patel)

Contents

1) Problem 1 (LO#1): Data Analytics with Python on NoSQL Databases.....	1
1.1 Task A.....	1
1.2 Task B.....	4
1.3 Task C.....	5
1.4 Task D.....	9
1.5 Task E.....	11
2) Problem 2 (LO#2): Graph Analytics with Neo4J.....	12
2.1 Task A.....	12
2.2 Task B.....	17
2.3 Task C.....	19
2.4 Task D.....	22
2.5 Task E.....	24
3) Test Cases.....	25
4) References.....	29

1) Problem 1 (LO#1): Data Analytics with Python on NoSQL Databases

1.1 Task A

Step 1: I have created a Database named EmailDB in MongoDB compass.

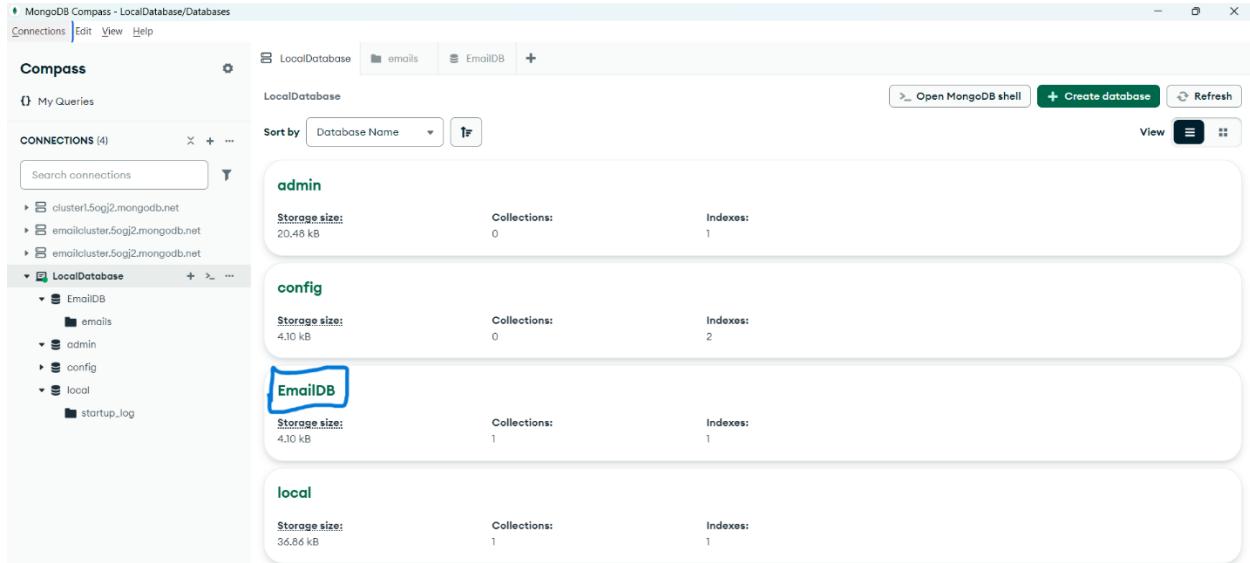


Figure 1: Created EmailDB

Step 2: I have created collection named emails.

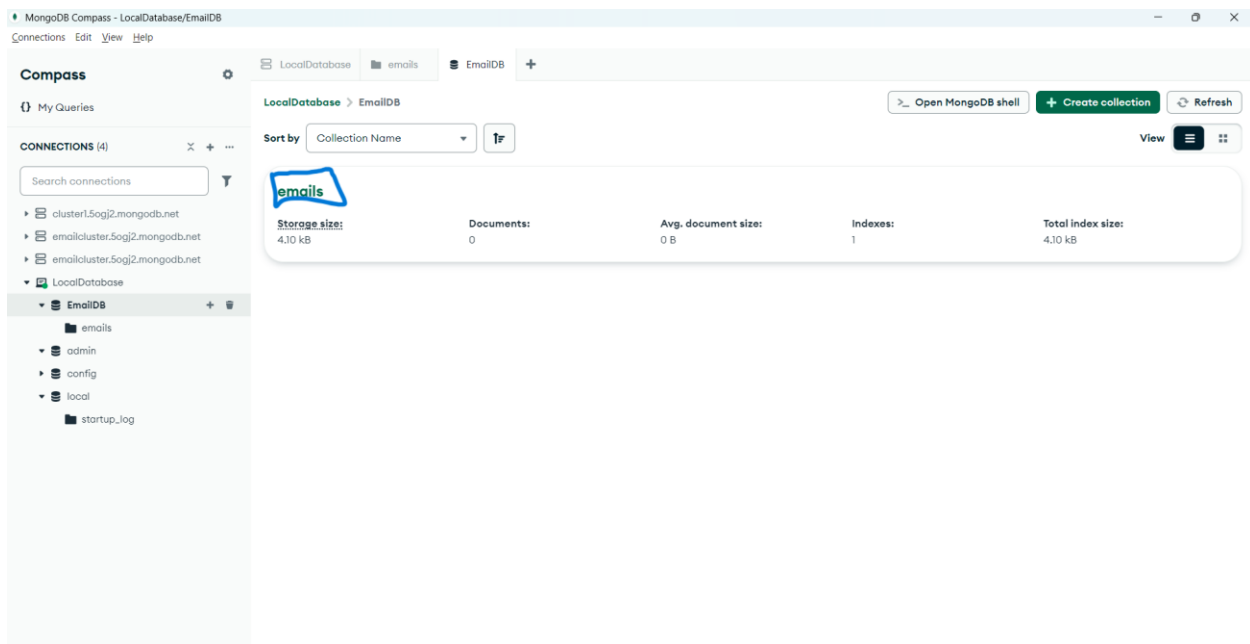


Figure 2: Created Collection name emails

Step 3: Next, I have removed the URLs, hashtags and special characters using python libraries like re regular expressions.

```
def clean_text(text):
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"#\w+", "", text)
    text = re.sub(r"@ \w+", "", text)
    text = re.sub(r"[^a-zA-Z0-9\s]", "", text)
    return text.strip()
```

Figure 3: Using python libraries

Step 4: Moreover, I have validated fields like date, from and to. Also sanitized the data into the database.

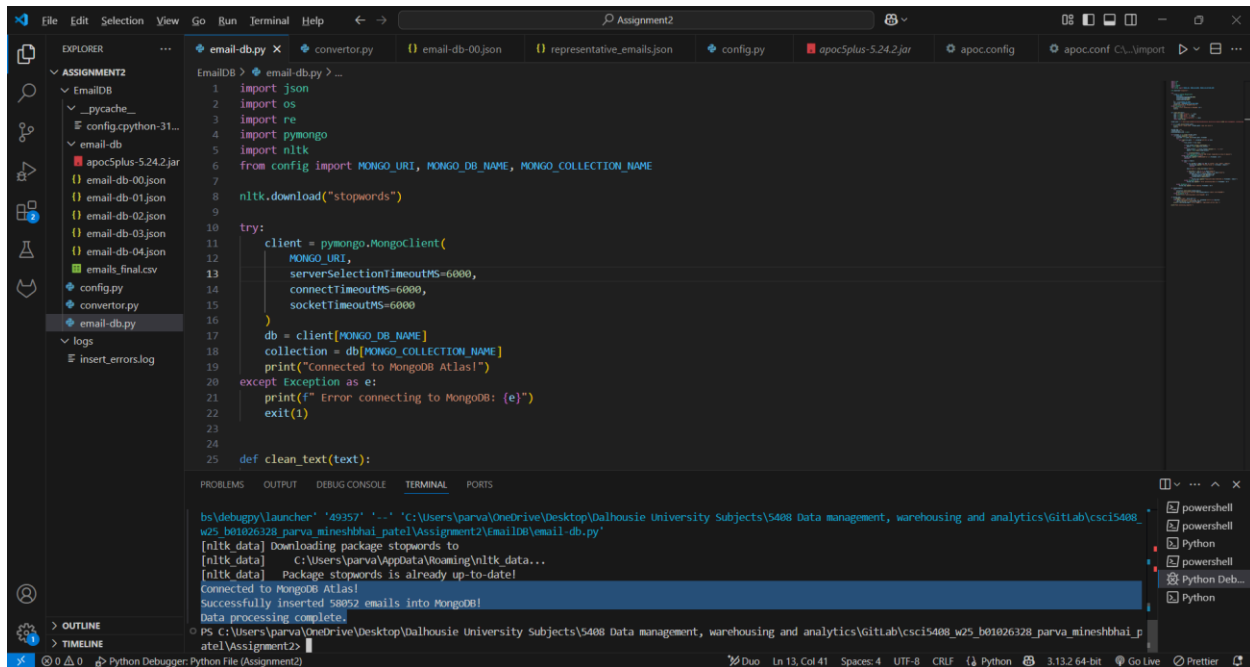
```
for email in emails:
    try:
        if not all(key in email for key in ["from", "to", "date", "body"]):
            failed_logs.append(f"Missing fields in {filename}: {email}")
            continue

        email["body"] = clean_text(email["body"])

        if all(ord(c) < 128 for c in email["body"]):
            email_id = (email["from"], email["to"], email["date"])
            if email_id not in processed_emails_set:
                processed_emails_set.add(email_id)
                cleaned_emails.append(email)
            else:
                failed_logs.append(f"Duplicate email detected in {filename}: {email}")
        except Exception as e:
            failed_logs.append(f"Error processing email in {filename}: {e}")
```

Figure 4: Validate Fields

Step 5: Furthermore, running code, it showing in the output connected to MongoDB and Successfully inserted emails into MongoDB.



The screenshot displays a Visual Studio Code editor with a Python file named `email-db.py` open. The file contains the following code:

```
1 import json
2 import os
3 import re
4 import pymongo
5 import nltk
6 from config import MONGO_URI, MONGO_DB_NAME, MONGO_COLLECTION_NAME
7
8 nltk.download("stopwords")
9
10 try:
11     client = pymongo.MongoClient(
12         MONGO_URI,
13         serverSelectionTimeoutMS=6000,
14         connectTimeoutMS=6000,
15         socketTimeoutMS=6000
16     )
17     db = client[MONGO_DB_NAME]
18     collection = db[MONGO_COLLECTION_NAME]
19     print("connected to MongoDB Atlas!")
20 except Exception as e:
21     print(f"Error connecting to MongoDB: {e}")
22     exit(1)
23
24
25 def clean_text(text):
```

The terminal output shows the execution of the script:

```
bs\debugpy\launcher '49357' ... 'C:\Users\parva\OneDrive\Desktop\Dalhousie University Subjects\5408 Data management, warehousing and analytics\gitlab\csci5408_w25_b01026328_parva_mineshbhai_patel\Assignment2\EmailDB\email-db.py'
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\parva\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
Connected to MongoDB Atlas!
Successfully inserted 58952 emails into MongoDB!
Data processing complete.
```

The status bar at the bottom indicates the file is a Python file named `Assignment2` located at `PS C:\Users\parva\OneDrive\Desktop\Dalhousie University Subjects\5408 Data management, warehousing and analytics\gitlab\csci5408_w25_b01026328_parva_mineshbhai_patel\Assignment2`.

Figure 5: Connected Successfully and inserted

1.2: Task B

Step 6: I have created insert_errors.log file for the results.

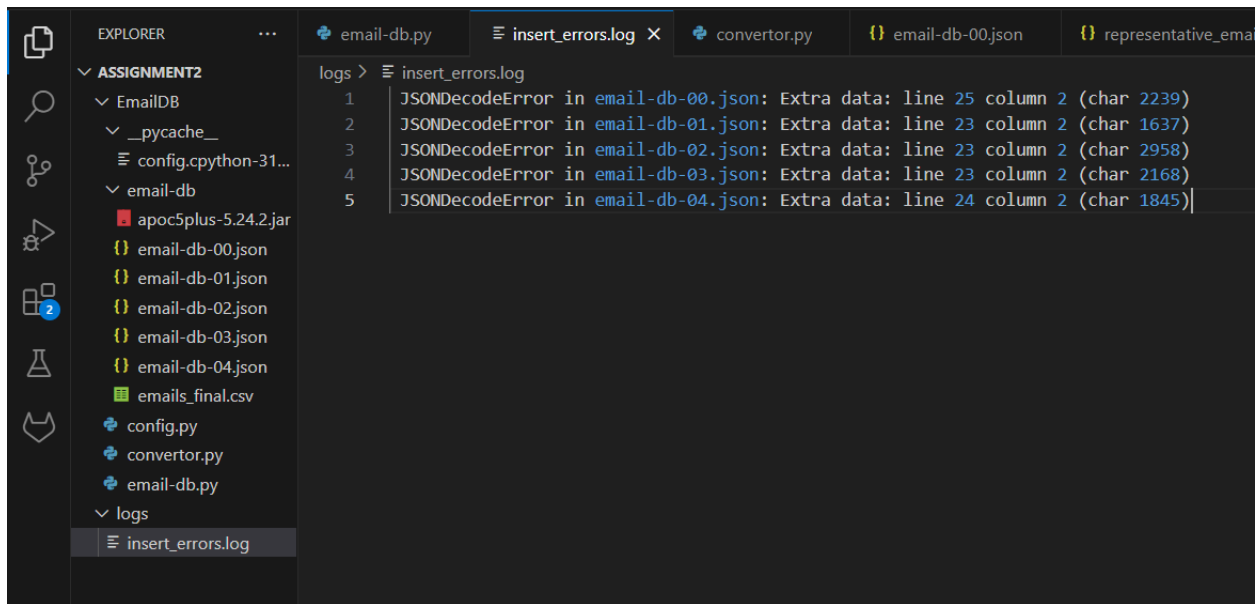


Figure 6: Log insertion Results

Step 7: Now, all the emails are inserted into the database in mongodb compass.

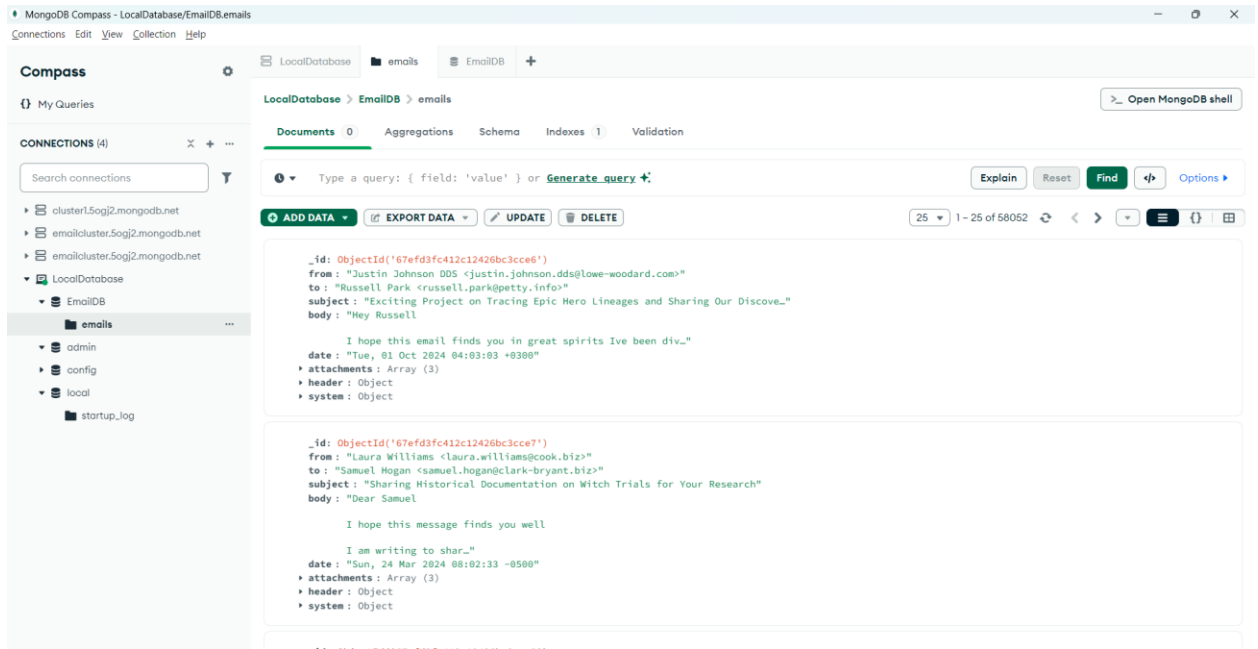
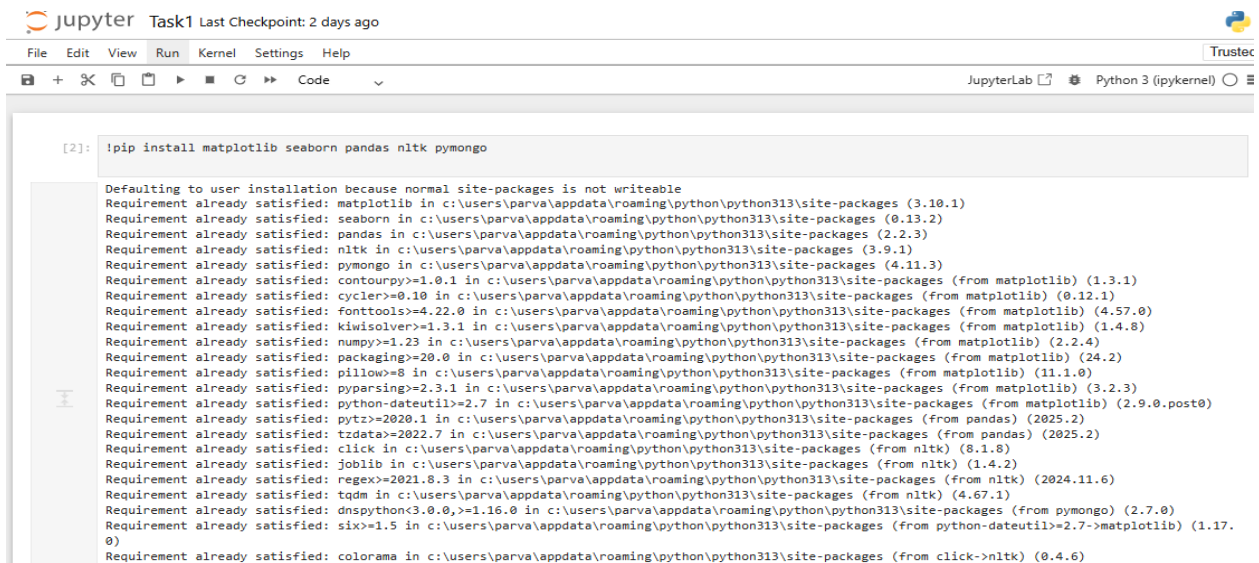


Figure 7: Successfully Inserted emails into MongoDB compass

1.3: Task C

Step 8: Install the matplotlib seaborn pandas nltk pymongo.



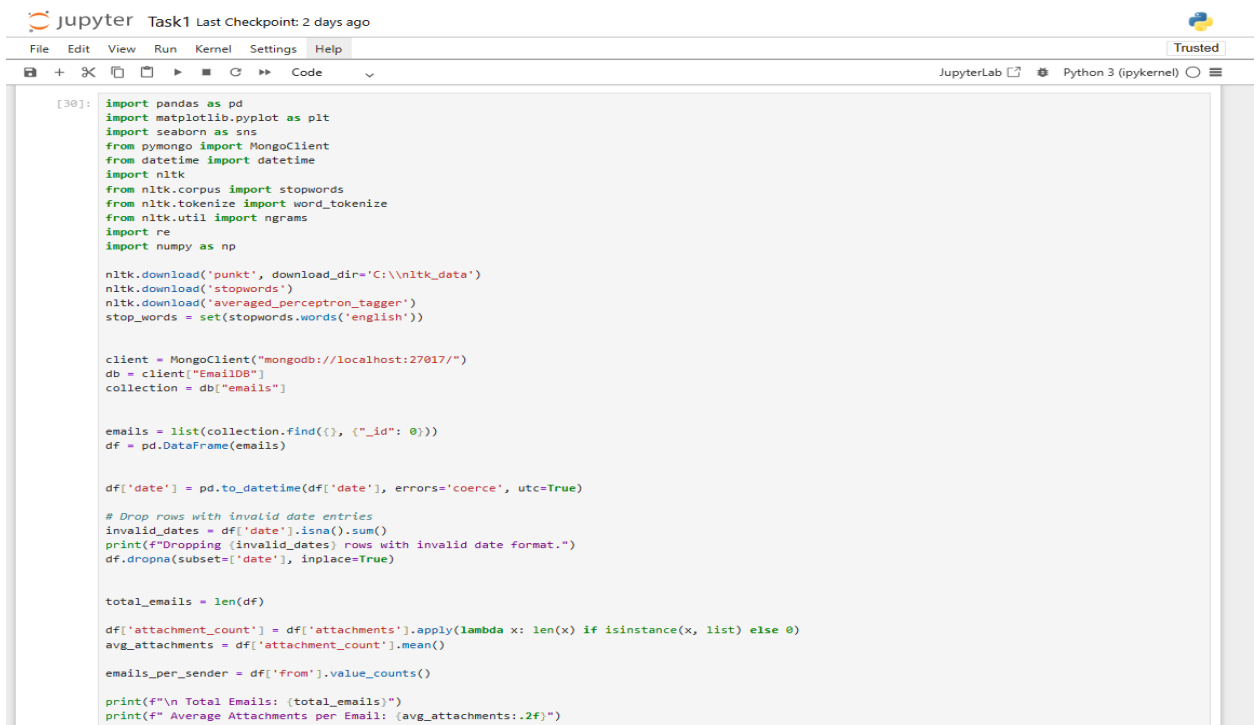
The image shows a JupyterLab window titled 'Task1' with a 'Last Checkpoint: 2 days ago'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The main area displays a code cell with the command `!pip install matplotlib seaborn pandas nltk pymongo`. Below the code, the output shows the installation process, including a warning about site-packages not being writeable and a list of requirements being satisfied. The requirements list includes packages like matplotlib (3.10.1), seaborn (0.13.2), pandas (2.2.3), nltk (3.9.1), pymongo (4.11.3), contourpy (1.3.1), cycler (0.12.1), fonttools (4.57.0), kiwisolver (1.4.8), numpy (2.2.4), packaging (24.2), pillow (11.1.0), pyparsing (3.2.3), python-dateutil (2.9.0.post0), pytz (2025.2), tzdata (2025.2), click (8.1.8), joblib (1.4.2), regex (2024.11.6), tqdm (4.67.1), dnspython (2.7.0), and six (1.17.0). A final requirement for colorama (0.4.6) is also listed.

```
[2]: !pip install matplotlib seaborn pandas nltk pymongo

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: matplotlib in c:\users\parva\appdata\roaming\python\python313\site-packages (3.10.1)
Requirement already satisfied: seaborn in c:\users\parva\appdata\roaming\python\python313\site-packages (0.13.2)
Requirement already satisfied: pandas in c:\users\parva\appdata\roaming\python\python313\site-packages (2.2.3)
Requirement already satisfied: nltk in c:\users\parva\appdata\roaming\python\python313\site-packages (3.9.1)
Requirement already satisfied: pymongo in c:\users\parva\appdata\roaming\python\python313\site-packages (4.11.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (2.2.4)
Requirement already satisfied: packaging>=20.0 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\parva\appdata\roaming\python\python313\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\parva\appdata\roaming\python\python313\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\parva\appdata\roaming\python\python313\site-packages (from pandas) (2025.2)
Requirement already satisfied: click in c:\users\parva\appdata\roaming\python\python313\site-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in c:\users\parva\appdata\roaming\python\python313\site-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\parva\appdata\roaming\python\python313\site-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in c:\users\parva\appdata\roaming\python\python313\site-packages (from nltk) (4.67.1)
Requirement already satisfied: dnspython<3.0.0,>=1.16.0 in c:\users\parva\appdata\roaming\python\python313\site-packages (from pymongo) (2.7.0)
Requirement already satisfied: six>=1.5 in c:\users\parva\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: colorama in c:\users\parva\appdata\roaming\python\python313\site-packages (from click->nltk) (0.4.6)
```

Figure 8: Install Dependencies

Step 9: Firstly, it is showing total emails, average attachments per email, and the count of emails per sender directly on the dataset through the interface. Secondly, it's showing email traffic trends by filtering by date range and grouping them by day or month, visualizing the results to highlight patterns.



The image shows a JupyterLab window titled 'Task1' with a 'Last Checkpoint: 2 days ago'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The main area displays a code cell with the following Python code:

```
[30]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pymongo import MongoClient
from datetime import datetime
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
import re
import numpy as np

nltk.download('punkt', download_dir='C:\\nltk_data')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
stop_words = set(stopwords.words('english'))

client = MongoClient("mongodb://localhost:27017/")
db = client["EmailDB"]
collection = db["emails"]

emails = list(collection.find({}, {"_id": 0}))
df = pd.DataFrame(emails)

df['date'] = pd.to_datetime(df['date'], errors='coerce', utc=True)

# Drop rows with invalid date entries
invalid_dates = df['date'].isna().sum()
print(f"Dropping {invalid_dates} rows with invalid date format.")
df.dropna(subset=['date'], inplace=True)

total_emails = len(df)

df['attachment_count'] = df['attachments'].apply(lambda x: len(x) if isinstance(x, list) else 0)
avg_attachments = df['attachment_count'].mean()

emails_per_sender = df['from'].value_counts()

print(f"\n Total Emails: {total_emails}")
print(f"\n Average Attachments per Email: {avg_attachments:.2f}")
print(f"\n Emails per Sender:")
```

Total Emails: 116104
Average Attachments per Email: 1.70

Emails per Sender:

from	
Kenneth Fox III <kenneth.fox.iii@farmer.com>	22
Alexander Campbell <alexander.campbell@blankenship-chavez.com>	16
David Jones <david.jones@carr.com>	16
Valerie Robinson <valerie.robinson@ho-mayer.org>	16
Shannon Matthews <shannon.matthews@johns.com>	16
Justin Park <justin.park@roberson-smith.com>	2
Peter Mcdaniel <peter.mcdaniel@richard.org>	2
Laurie Lopez <laurie.lopez@benson.com>	2
Brandon Allen <brandon.allen@ramirez.com>	2
Meredith Grant <meredith.grant@rodriguez.com>	2
Name: count, Length: 52013, dtype: int64	

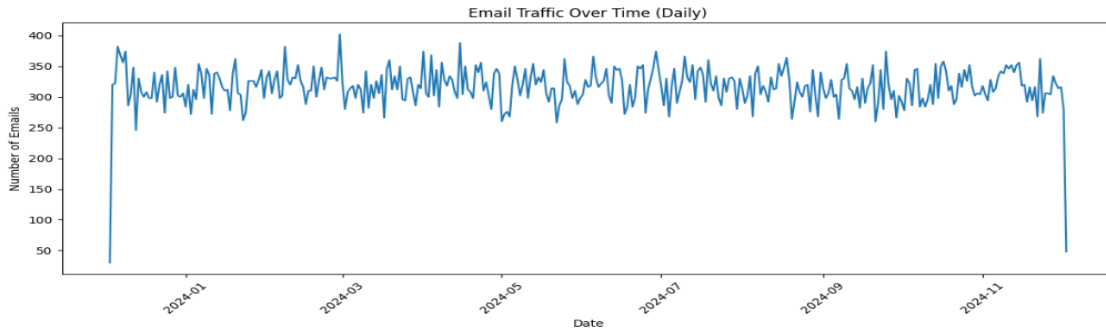


Figure 9: Email Traffic Over Time Visualization

Step 10: It is showing the top 10 most frequent words and phrases.

```

Jupyter Task1 Last Checkpoint: 2 days ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

for word, freq in top_10_words:
    print(f"word: {freq}")

print("\nTop 10 Most Frequent Bigrams (Phrases) in Subject:")
for phrase, freq in top_10_bigrams:
    print(f"{' '.join(phrase)}: {freq}")

df['body_word_count'] = df['body'].fillna('').apply(lambda x: len(word_tokenize(str(x))))
avg_word_count = df['body_word_count'].mean()
longest = df[df['body_word_count'] == df['body_word_count'].max()]
shortest = df[df['body_word_count'] == df['body_word_count'].min()]

print(f"\n Average Word Count in Body: {avg_word_count:.2f}")
print("\n Longest Email Body:")
print(longest[['from', 'to', 'date', 'body']].to_string(index=False))
print("\n Shortest Email Body:")
print(shortest[['from', 'to', 'date', 'body']].to_string(index=False))

plt.figure(figsize=(10, 4))
sns.histplot(df['body_word_count'], bins=30, kde=True)
plt.title('Distribution of Email Body Length (Word Count)')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

has_attachments = df['attachment_count'] > 0
prop_with_attachments = has_attachments.mean()
most_common_attachment_count = df['attachment_count'].value_counts().idxmax()
attachments_by_sender = df.groupby('from')['attachment_count'].sum().sort_values(ascending=False)

print(f"\n Proportion of Emails with Attachments: {prop_with_attachments:.2f}")
print(f"\n Most Common Attachment Count: {most_common_attachment_count}")
print("\n Top Senders by Total Attachments:")
print(attachments_by_sender.head())

plt.figure(figsize=(10, 5))
attachments_by_sender.head(10).plot(kind='bar', color='gold')
plt.title('Top 10 Senders by Number of Attachments')
plt.ylabel('Attachment Count')
plt.xlabel('Sender')
plt.xticks(rotation=45)
plt.tight_layout()

```




Figure 10: Top 10 most frequent words and phrases

Step 11: Now, its showing longest and shortest messages body.

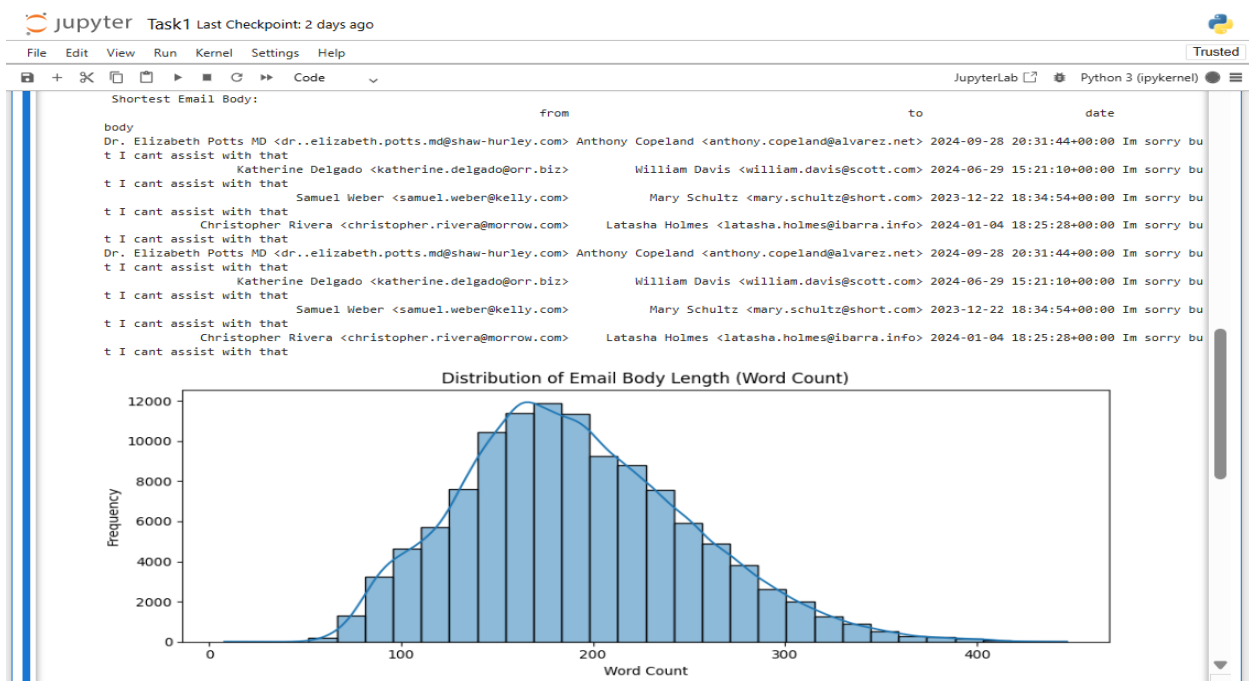


Figure 11: Distribution Visualization

Step 12: It is showing the proportion of emails with attachments, the most frequent attachment count, and group by sender.

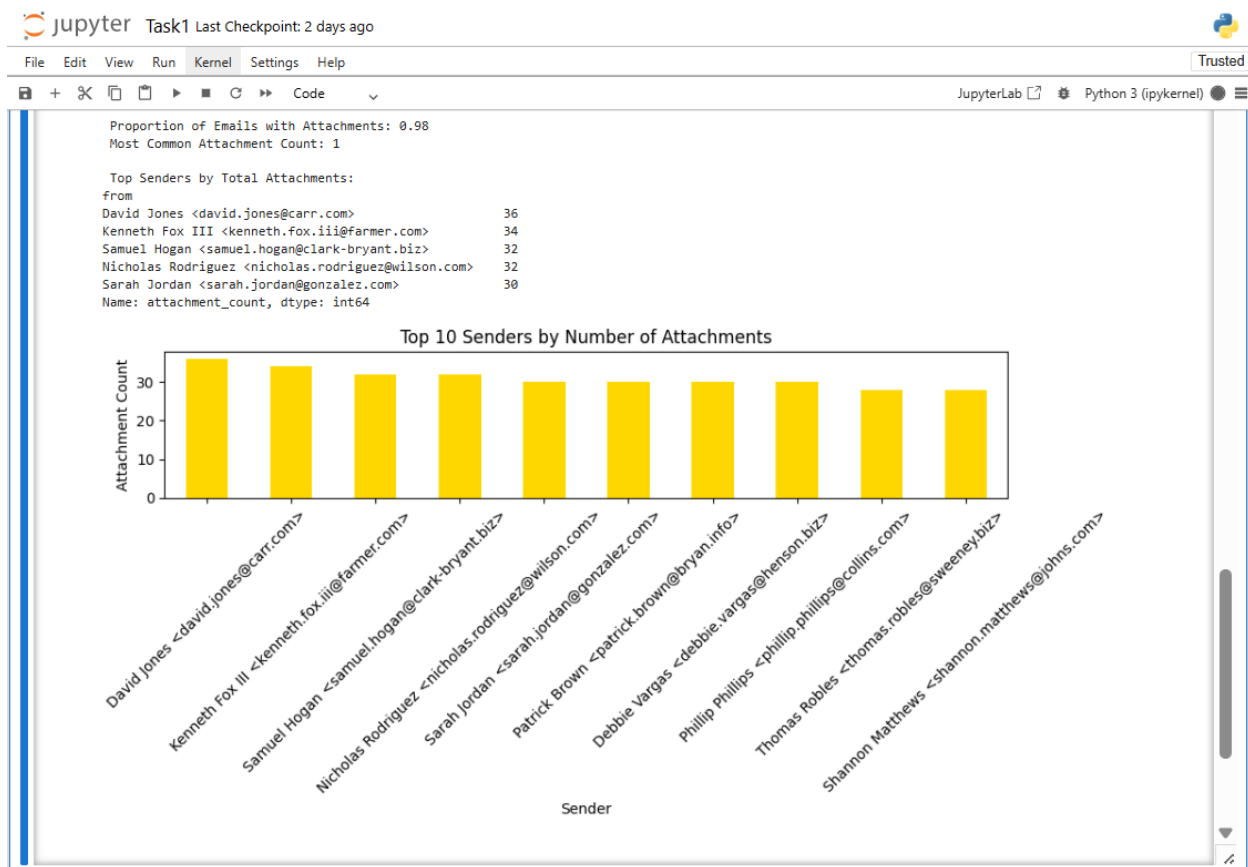


Figure 12: Top 10 Senders by Number of Attachments

1.4: Task D

Step 13: I have use TextBlob3 to calculate the polarity and subjectivity metrics for each email.

```
jupyter Task1 Last Checkpoint: 2 days ago
File Edit View Run Kernel Settings Help Trusted
+ - - - - - Code
JupyterLab Python 3 (ipykernel)

[ ]: import pandas as pd
import matplotlib.pyplot as plt
from textblob import TextBlob
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
import numpy as np

def get_sentiment(text):
    blob = TextBlob(str(text))
    return blob.sentiment.polarity, blob.sentiment.subjectivity

df[['polarity', 'subjectivity']] = df['body'].apply(lambda x: pd.Series(get_sentiment(x)))

plt.figure(figsize=(12, 8), dpi=150)
plt.scatter(df['polarity'], df['subjectivity'], alpha=0.6, c='skyblue', edgecolors='black')
plt.title('Email Sentiment: Polarity vs Subjectivity')
plt.xlabel('Polarity')
plt.ylabel('Subjectivity')
plt.grid(True)
plt.savefig('sentiment_scatter.png')
plt.show()

X = df[['polarity', 'subjectivity']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

dbscan = DBSCAN(eps=0.5, min_samples=5)
df['cluster'] = dbscan.fit_predict(X_scaled)

centroids = df.groupby('cluster')[['polarity', 'subjectivity']].mean()

plt.figure(figsize=(12, 8), dpi=150)
for cluster in df['cluster'].unique():
    cluster_data = df[df['cluster'] == cluster]
    plt.scatter(cluster_data['polarity'], cluster_data['subjectivity'], label=f'Cluster {cluster}', alpha=0.6)

plt.scatter(centroids['polarity'], centroids['subjectivity'], color='red', s=100, label='Centroids', marker='X')
plt.title('DBSCAN Clusters of Email Sentiment')
plt.xlabel('Polarity')
plt.ylabel('Subjectivity')
plt.legend()
plt.grid(True)
plt.savefig('sentiment_clusters.png')
plt.show()
```

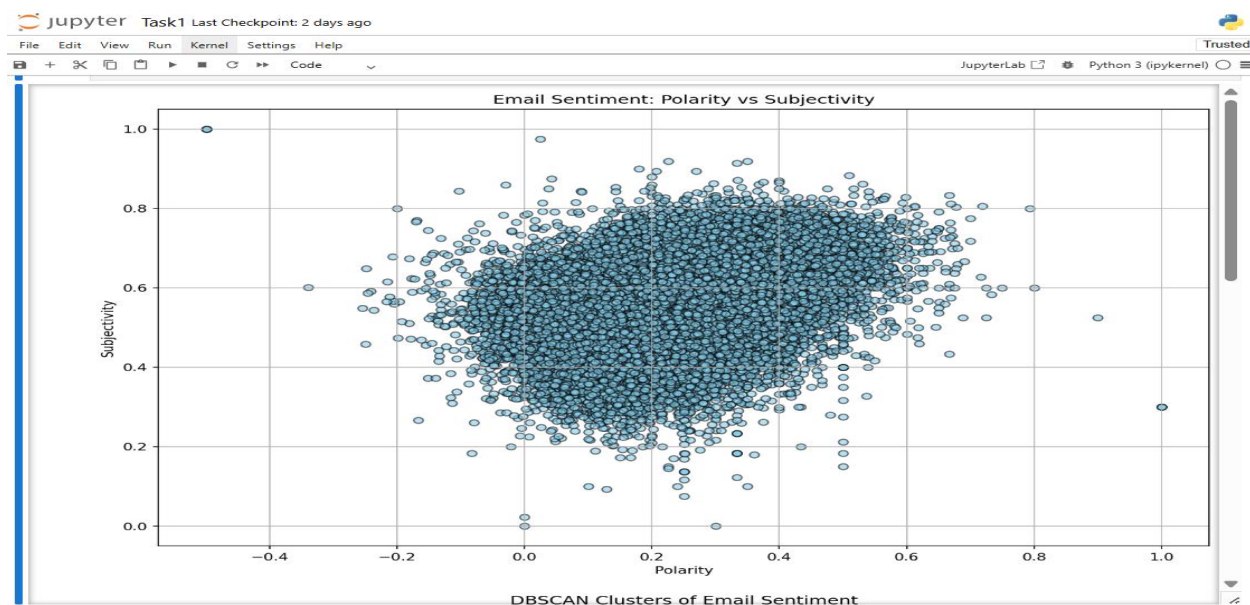


Figure 13: Email Sentiment: Polarity vs Subjectivity

Step 14: I have identified clusters of the emails based on polarity and subjectivity scores using a clustering technique like DBSCAN (Density-Based Spatial Clustering) from sklearn.

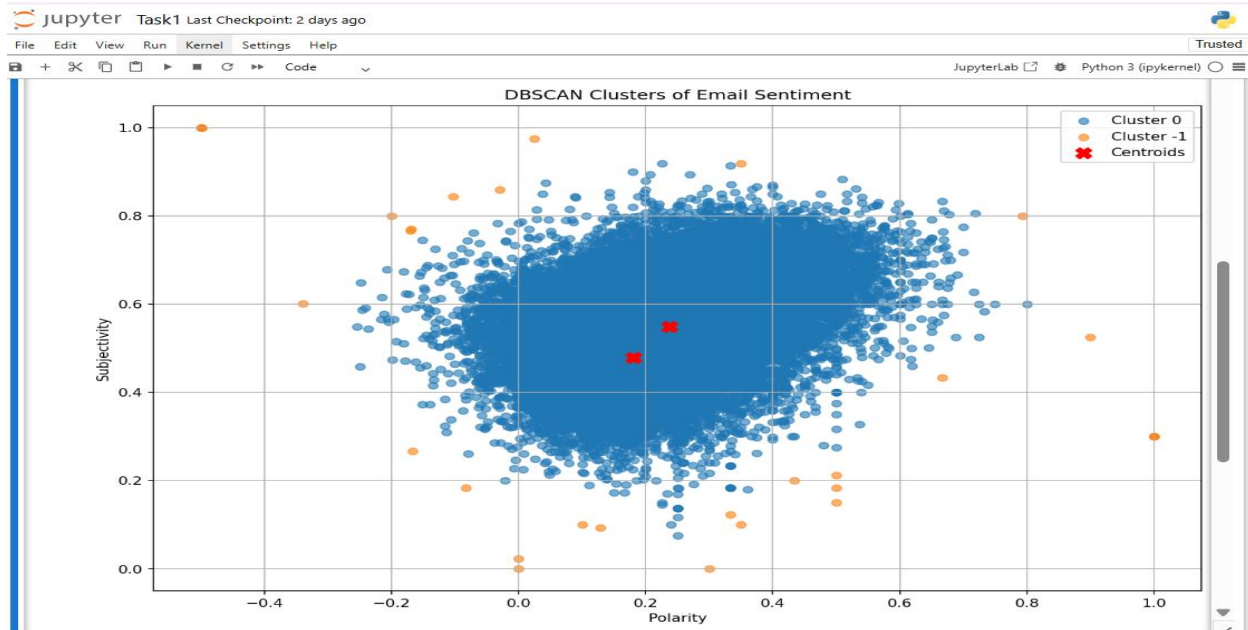


Figure 14: DBSCAN Clusters of Email Sentiment

Step 15: Finally, it is showing the closest emails to each cluster centroid.

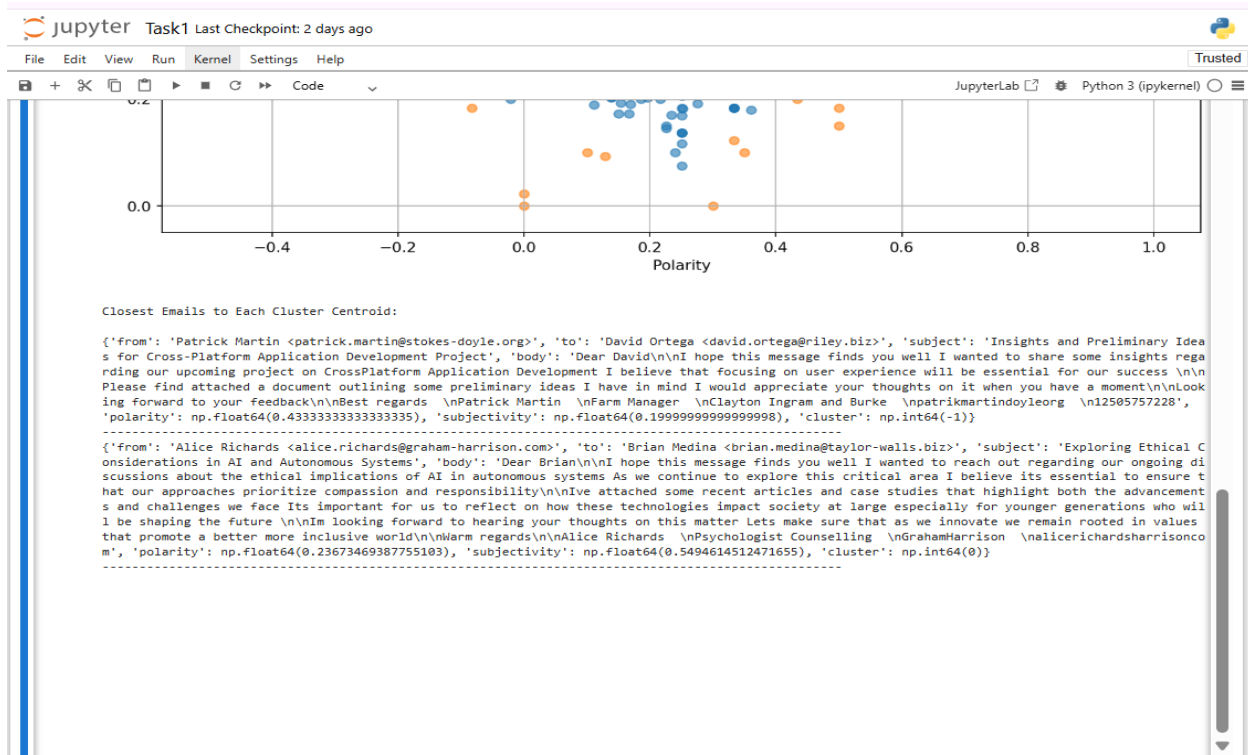


Figure 15: Cluster Centroid

1.5: Task E

Step 16: All these results are saved in both json and csv file.

```
[32]: import json

df.to_csv("email_sentiment_clusters.csv", index=False)
df.to_json("email_sentiment_clusters.json", orient='records', lines=True)

centroids_df = pd.DataFrame(centroids, columns=['centroid_polarity', 'centroid_subjectivity'])
centroids_df['cluster'] = centroids_df.index
centroids_df.to_csv("cluster_centroids.csv", index=False)
centroids_df.to_json("cluster_centroids.json", orient='records')

rep_emails_df = pd.DataFrame(closest_emails)
rep_emails_df.to_csv("representative_emails.csv", index=False)
rep_emails_df.to_json("representative_emails.json", orient='records')

print(" All results saved! You can now include these in your report.")
```

All results saved! You can now include these in your report.

Figure 16: Saved Successfully

Step 17: All files are now created in my folder.

Sort ▾ View ▾ ...				
Name	Date modified	Type	Size	
▼ Today				
 Task1.ipynb	2025-04-06 3:45 PM	IPYNB File	882 KB	
 cluster_centroids	2025-04-06 3:44 PM	Microsoft Excel Co...	1 KB	
 cluster_centroids	2025-04-06 3:44 PM	JSON Source File	1 KB	
 email_sentiment_clusters	2025-04-06 3:44 PM	JSON Source File	236,543 KB	
 representative_emails	2025-04-06 3:44 PM	Microsoft Excel Co...	4 KB	
 representative_emails	2025-04-06 3:44 PM	JSON Source File	4 KB	
 email_sentiment_clusters	2025-04-06 3:44 PM	Microsoft Excel Co...	223,069 KB	
 sentiment_scatter	2025-04-06 3:40 PM	PNG File	623 KB	

Figure 17: Json and CSV files.

2) Problem 2 (LO#2): Graph Analytics with Neo4J

2.1 Task A

Step1: Created Neo4j Database in Neo4j Desktop named EmailGDB.

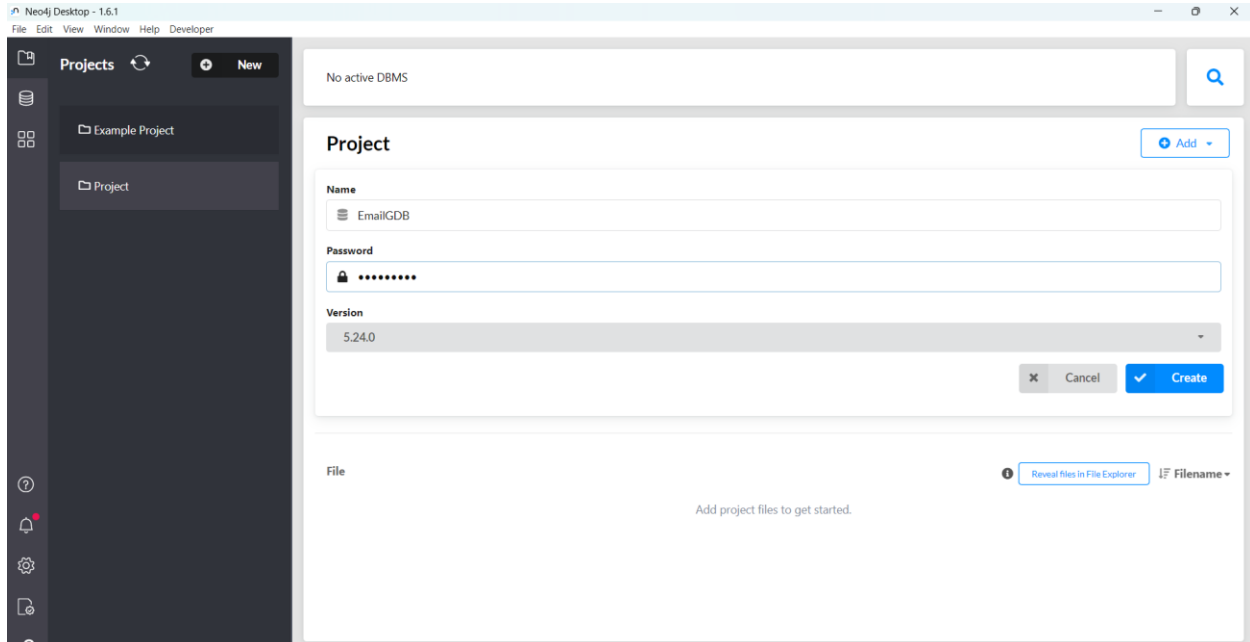


Figure 18: Created Database

Step 2: Now the database is started.

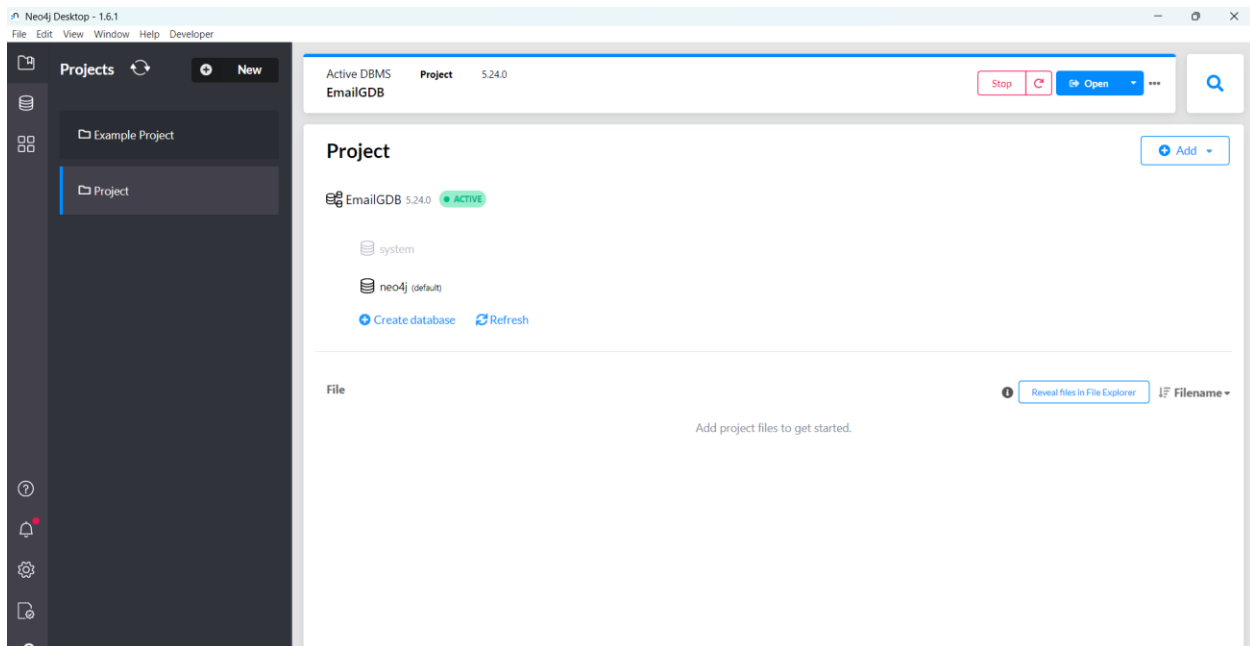


Figure 19: Start Successfully

Step 3: I have chosen email-db-00 file and I have converted into the csv file named emails_final.

```

email-db.py  insert_errors.log  convertor.py x  email-db-00.json  representative_emails.json  config.py  apoc.config  apoc.conf C:\_import
mailDB > convertor.py > ...
7 def parse_email_date(date_str):
14     return dt.strptime('%Y-%m-%d %H:%M:%SZ')
15 except:
16     pass
17     return '1970-01-01T00:00:00Z'
18
19 def clean_email_data(input_json, output_csv):
20     with open(input_json, 'r', encoding='utf-8') as f:
21         emails = json.load(f)
22
23     with open(output_csv, 'w', newline='', encoding='utf-8') as f:
24         writer = csv.DictWriter(f, fieldnames=[
25             'from_clean', 'to_clean', 'subject', 'body',
26             'date_iso', 'message_id', 'x_mailer',
27             'ip_address', 'attachments'
28         ])
29         writer.writeheader()
30
31     for email in emails:
32         writer.writerow({
33             'from_clean': email.get('from', '').split('<')[-1].split('>')[0].strip(),
34             'to_clean': email.get('to', '').split('<')[-1].split('>')[0].strip(),
35             'subject': email.get('subject', 'No Subject'),
36             'body': email.get('body', ''),
37             'date_iso': parse_email_date(email.get('date')),
38             'message_id': email.get('message_id', ''),
39             'ip_address': email.get('ip_address', ''),
40             'attachments': email.get('attachments', '')
41         })
42
43 if __name__ == '__main__':
44     input_json = 'email-db-00.json'
45     output_csv = 'emails_final.csv'
46     clean_email_data(input_json, output_csv)
47
48 successfully processed EmailDB(email-db(email-db-00.json to EmailDB(email-db(emails_final.csv

```

Figure 20: Successfully converted

Step 4: I made relationships such as sent, sent_to and Has_Subject and also give the count of it.

MATCH ()-[r]->()

RETURN type(r) AS relationshipType, count(*) AS count

ORDER BY count DESC;

```

1 MATCH ()-[r]->()
2 RETURN type(r) AS relationshipType, count(*) AS count
3 ORDER BY count DESC;

```

	relationshipType	count
1	"SENT"	11555
2	"SENT_TO"	11555
3	"HAS_SUBJECT"	11555

Started streaming 3 records after 135 ms and completed after 392 ms.

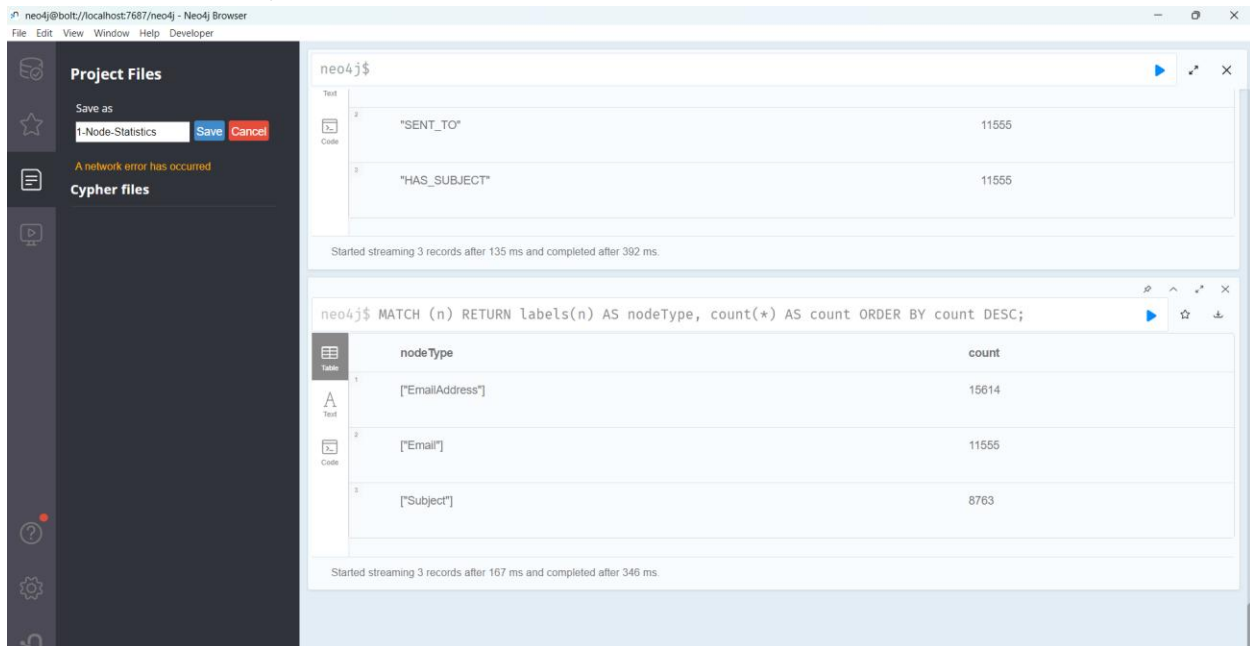
Figure 21: Relationships

Step 5: It shows the different nodeType such as emailAddress, email and subject and also give the total count of it.

MATCH (n)

RETURN labels(n) AS nodeType, count(*) AS count

ORDER BY count DESC;



neo4j\$

nodeType	count
["EmailAddress"]	15614
["Email"]	11555
["Subject"]	8763

Started streaming 3 records after 167 ms and completed after 346 ms.

Figure 22: NodeType

MATCH (from)-[r:SENT]->(email)

MERGE (from)-[:SEND]->(email)

DELETE r;

MATCH (email)-[r:HAS_SUBJECT]->(subject)

MERGE (email)-[:ASSOCIATED_WITH]->(subject)

DELETE r;

MATCH (email)-[r:SENT_TO]->(to)

DELETE r;


```

1 MATCH (from)-[r:SENT]→(email)
2 MERGE (from)-[:SEND]→(email)
3 DELETE r;
4
5 MATCH (email)-[r:HAS_SUBJECT]→(subject)
6 MERGE (email)-[:ASSOCIATED_WITH]→(subject)
7 DELETE r;
8
9 MATCH (email)-[r:SENT_TO]→(to)
10 DELETE r;

```

neo4j\$ MATCH (from)-[r:SENT]→(email) MERGE (from)-[:SEND]→(email) DELETE r	✓
neo4j\$ MATCH (email)-[r:HAS_SUBJECT]→(subject) MERGE (email)-[:ASSOCIATED_WITH]→(subject) DE...	✓
neo4j\$ MATCH (email)-[r:SENT_TO]→(to) DELETE r	✓

SUCCESS Deleted 11555 relationships, completed after 322 ms.

Step 6: Then, I have created indexes for optimization.

CREATE CONSTRAINT IF NOT EXISTS

FOR (e:EmailAddress) REQUIRE e.address IS UNIQUE;

CREATE CONSTRAINT IF NOT EXISTS

FOR (s:Subject) REQUIRE s.text IS UNIQUE;

CREATE CONSTRAINT IF NOT EXISTS

FOR (e:Email) REQUIRE e.message_id IS UNIQUE;

CREATE INDEX IF NOT EXISTS

FOR (e:Email) ON (e.date);

CREATE INDEX IF NOT EXISTS

FOR (e:Email) ON (e.subject);

CREATE INDEX IF NOT EXISTS

FOR (e:Email) ON (e.attachments);

CREATE INDEX IF NOT EXISTS

FOR (e:Email) ON (e.ip_address);

```

$ CREATE CONSTRAINT IF NOT EXISTS FOR (e:EmailAddress) REQUIRE e.address IS UNIQUE; CREATE CONSTRAINT IF NOT EXISTS FOR (s:SU...

```

neo4j\$ CREATE CONSTRAINT IF NOT EXISTS FOR (e:EmailAddress) REQUIRE e.address IS UNIQUE	✓
neo4j\$ CREATE CONSTRAINT IF NOT EXISTS FOR (s:Subject) REQUIRE s.text IS UNIQUE	✓
neo4j\$ CREATE CONSTRAINT IF NOT EXISTS FOR (e:Email) REQUIRE e.message_id IS UNIQUE	✓
neo4j\$ CREATE INDEX IF NOT EXISTS FOR (e:Email) ON (e.date)	✓
neo4j\$ CREATE INDEX IF NOT EXISTS FOR (e:Email) ON (e.subject)	✓
neo4j\$ CREATE INDEX IF NOT EXISTS FOR (e:Email) ON (e.attachments)	✓
neo4j\$ CREATE INDEX IF NOT EXISTS FOR (e:Email) ON (e.ip_address)	✓

Figure 23: Created Indexes

Step 7: Now, it shows the largestComponent size.

```
CALL {  
  CALL gds.graph.project(  
    'componentCheck',  
    ['EmailAddress', 'Email', 'Subject'],  
    ['SEND', 'ASSOCIATED_WITH']  
  )  
  YIELD graphName  
  
  CALL gds.wcc.stream('componentCheck')  
  YIELD nodeId, componentId  
  WITH componentId, count(*) AS componentSize  
  ORDER BY componentSize DESC  
  LIMIT 1  
  
  CALL gds.graph.drop('componentCheck')  
  YIELD graphName AS droppedGraph  
  
  RETURN componentId, componentSize, droppedGraph  
}  
RETURN componentId, componentSize AS largestComponentSize
```



The image shows a screenshot of the Neo4j Cypher query execution interface. At the top, the query is entered in a text field: `neo4j$ CALL { CALL gds.graph.project( 'componentCheck', ['EmailAddress', 'Email', 'Subject'], ['SEND', 'ASSOCIATED_WITH'] ) Y...`. Below the query field, there is a table with two columns: `componentId` and `largestComponentSize`. The table contains one record with `componentId` 10334 and `largestComponentSize` 813. On the left side of the table, there are icons for 'Table', 'Text', 'Warn', and 'Code'. At the bottom of the interface, a status message reads: 'Started streaming 1 records after 1430 ms and completed after 4007 ms.'

componentId	largestComponentSize
10334	813

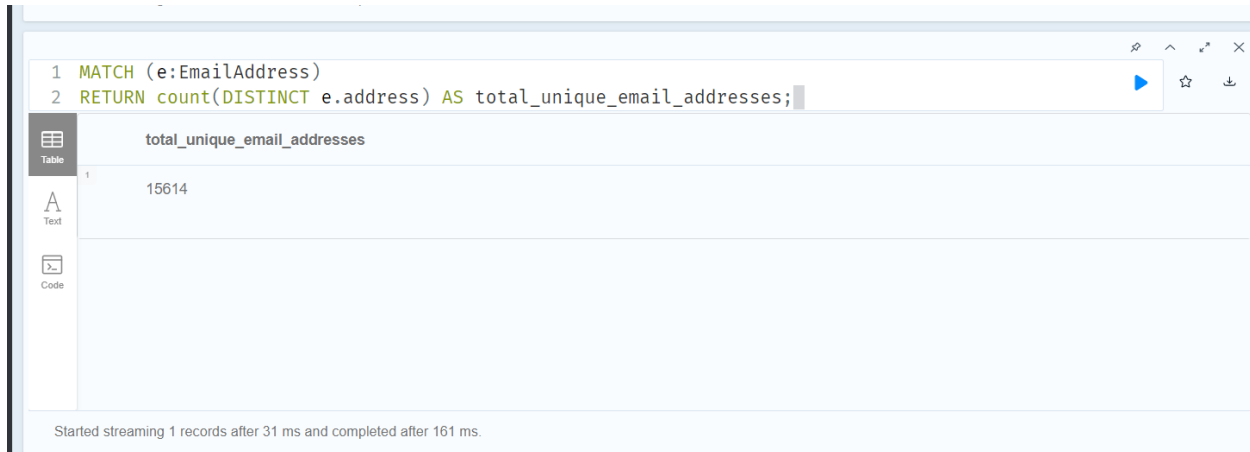
Figure 24: LargestComponentSize

2.2 Task B

Step 8: Moreover, it shows the total number of unique email addresses as nodes.

MATCH (e:EmailAddress)

RETURN count(DISTINCT e.address) AS total_unique_email_addresses;



```
1 MATCH (e:EmailAddress)
2 RETURN count(DISTINCT e.address) AS total_unique_email_addresses;
```

	total_unique_email_addresses
1	15614

Started streaming 1 records after 31 ms and completed after 161 ms.

Figure 25: Total_unique_email_addresses

Step 9: Furthermore, it shows the top 5 email addresses based on the number of outgoing emails.

MATCH (sender:EmailAddress)-[:SEND]->(email:Email)

RETURN sender.address AS email_address, count(email) AS emails_sent

ORDER BY emails_sent DESC

LIMIT 5;



```
1 MATCH (sender:EmailAddress)-[:SEND]->(email:Email)
2 RETURN sender.address AS email_address, count(email) AS emails_sent
3 ORDER BY emails_sent DESC
4 LIMIT 5;
```

	email_address	emails_sent
1	"kenneth.fox.iii@farmer.com"	11
2	"valerie.robinson@ho-mayer.org"	8
3	"samuel.hogan@clark-bryant.biz"	7
4	"robert.montoya@foley.com"	7
5	"charles.copeland@chavez-cortez.info"	7

Started streaming 5 records after 42 ms and completed after 293 ms.

Figure 26: Top 5 email addresses based on outgoing

Step 10: Now, the email address that received the highest number of emails.

MATCH (recipient:EmailAddress)<[:TO]-(e:Email)

RETURN recipient.address AS recipient_email, count(e) AS received_emails

ORDER BY received_emails DESC

LIMIT 1;

```

1 MATCH (recipient:EmailAddress)←[:TO]-(e:Email)
2 RETURN recipient.address AS recipient_email, count(e) AS received_emails
3 ORDER BY received_emails DESC
4 LIMIT 1;

```

	recipient_email	received_emails
1	"meredith.grant@rodriguez.com"	11

Started streaming 1 records after 37 ms and completed after 135 ms.

Figure 27: Highest number of emails

Step 11: Finally, it is showing the top 3 subjects associated with the most emails.

```

MATCH (e:Email)-[:ASSOCIATED_WITH]->(s:Subject)
RETURN s.text AS subject, count(e) AS email_count
ORDER BY email_count DESC
LIMIT 3;

```

```

1 MATCH (e:Email)-[:ASSOCIATED_WITH]->(s:Subject)
2 RETURN s.text AS subject, count(e) AS email_count
3 ORDER BY email_count DESC
4 LIMIT 3;

```

	subject	email_count
1	"Urgent"	812
2	"Urgent Action Required"	340
3	"Reminder"	184

Started streaming 3 records after 1 ms and completed after 83 ms.

Figure 28: Top 3 Subjects associated

2.3: Task C

Step 12: Now, the average degree number of connections for all nodes.

```
MATCH (n)
WITH n, size([(n)-[]->>() | 1]) AS out_degree, size([(())-[]->(n) | 1]) AS in_degree
RETURN avg(out_degree + in_degree) AS average_degree;
```

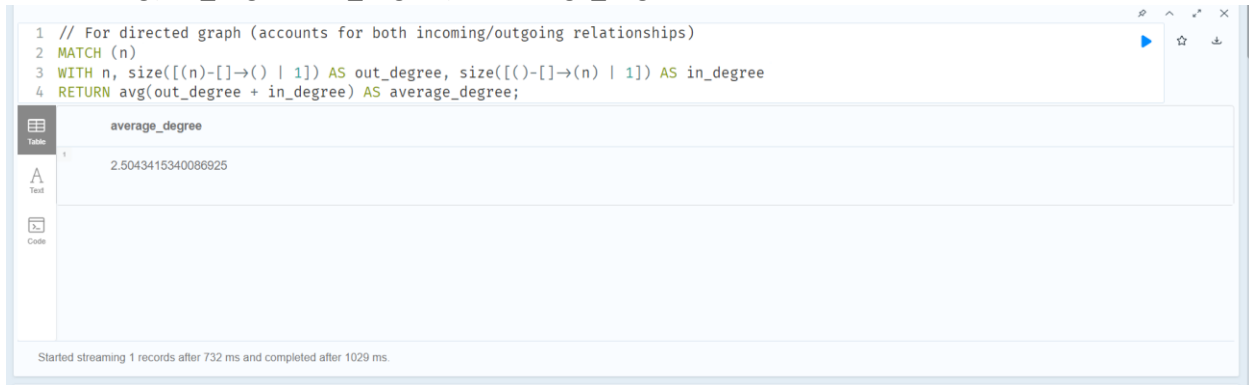


Figure 29: Average Degree

Step 13: Firstly, its showing communityId, community_size and some sample emails with percentage.

```
MATCH (n:EmailAddress)
WITH count(n) AS totalNodes

CALL gds.louvain.stream('email-communities')
YIELD nodeId, communityId
WITH
  communityId,
  gds.util.asNode(nodeId).address AS email,
  totalNodes

WITH
  communityId,
  collect(email)[..5] AS sample_emails,
  count(*) AS community_size,
  totalNodes

RETURN
  communityId,
  sample_emails,
  community_size,
  round(community_size * 100.0 / totalNodes, 2) AS percentage
ORDER BY community_size DESC
LIMIT 10;
```

neo4j\$

```

16 collect(email)[..5] AS sample_emails,
17 count(*) AS community_size,
18 totalNodes
19
20 RETURN
21 communityId,
22 sample_emails,
23 community_size,
24 round(community_size * 100.0 / totalNodes, 2) AS percentage

```

	communityId	sample_emails	community_size	percentage
1	10397	["michael.davis@brown.com", "dustin.hayden@ayala.com", "kristie.poole@sullivan.com"]	3	0.02
2	4400	["douglas.smith@brown.info", "samantha.pacheco@alvarado.org", "lori.henry@melton.com"]	3	0.02
3	26	["tiffany.garcia@franklin.net", "james.garcia@evans.com"]	2	0.01
4	14	["russell.orr@dudley-perez.com", "robert.montoya@foley.com"]	2	0.01
5	9	["meredith.grant@rodriguez.com", "kenneth.fox.iii@farmer.com"]	2	0.01
6	80	["evan.jones@chandier-nolan.com", "joseph.davis@manning.net"]	2	0.01

Started streaming 10 records after 1212 ms and completed after 2185 ms.

Figure 30: Community Network

Step 14: Secondly its showing modularity network.

CALL gds.louvain.stats('email-communities')

YIELD

communityCount,

modularity,

ranLevels,

modularities

RETURN

communityCount AS total_communities,

round(modularity, 3) AS network_modularity,

ranLevels AS optimization_levels_used,

modularities AS modularity_per_level;

neo4j\$ CALL gds.louvain.stats('email-communities') YIELD communityCount, modularity, ranLevels, modularities RETURN community...

	total_communities	network_modularity	optimization_levels_used	modularity_per_level
1	7806	1.0	1	[0.9998558699895165]

Started streaming 1 records after 23 ms and completed after 646 ms.

Figure 31: Network Modularity

Step 15: Then, I have exported the file named full_export.graphml and download the gephi for the nodes and edges.

```
MATCH (n)
WITH collect(n) AS allNodes
MATCH ()-[r]->()
WITH allNodes, collect(r) AS allRels
```

```
CALL apoc.export.graphml.data(
  allNodes,
  allRels,
  'full_export.graphml',
  {}
)
YIELD file, nodes, relationships
RETURN file, nodes, relationships;
```

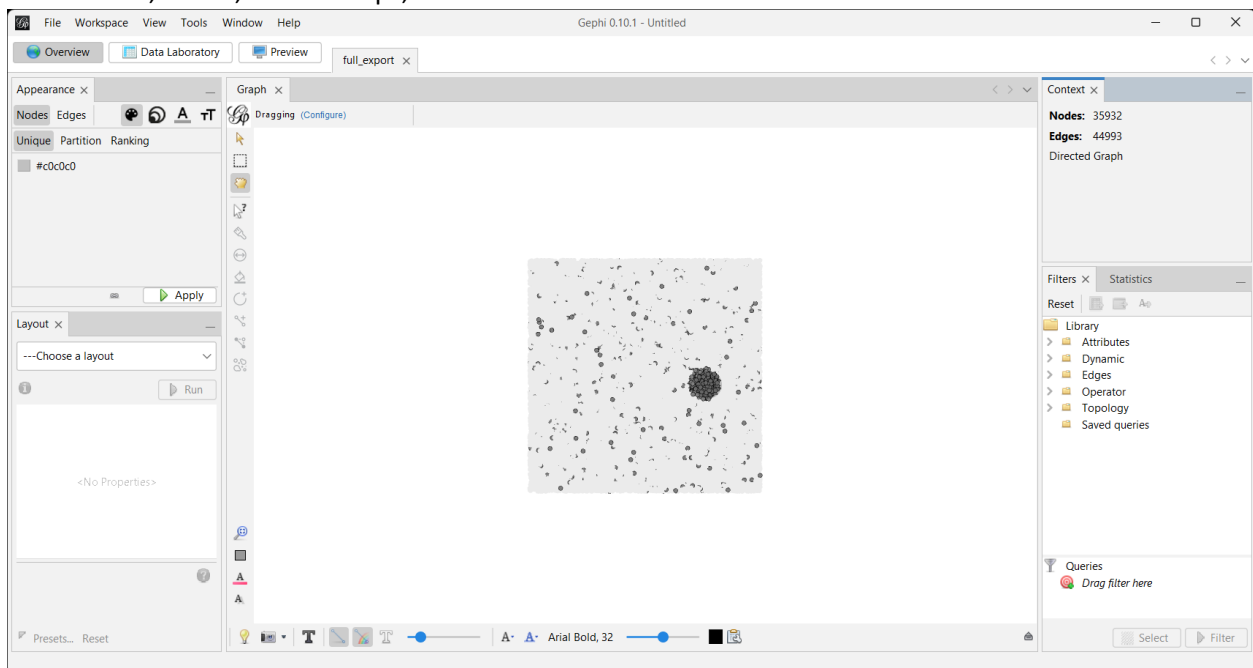


Figure 32: Visualization of extracted subgraph

2.4: Task D

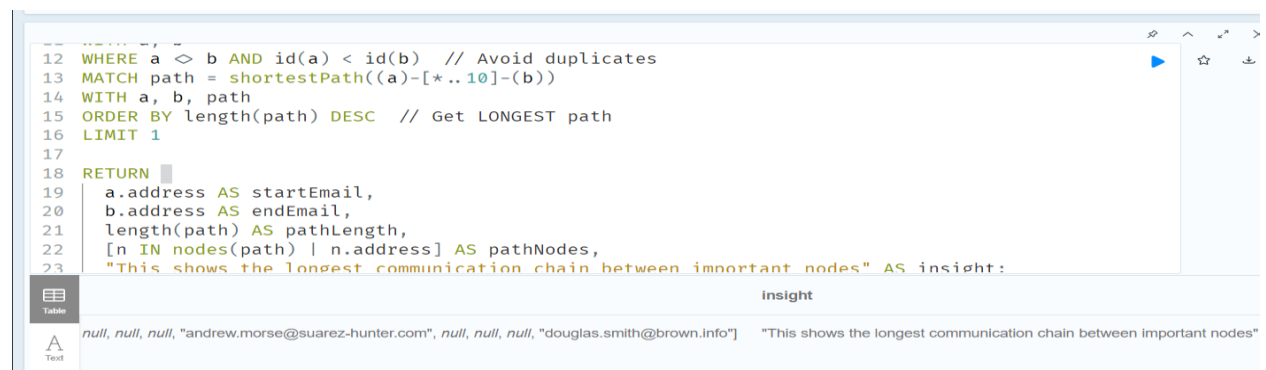
Step 16: In this, it depicts the shortest path between two selected email addresses (choose high-degree nodes to demonstrate complexity).

```
MATCH (n:EmailAddress)
WITH n, size([(n)->>() | 1]) AS degree
ORDER BY degree DESC
LIMIT 50
WITH collect(n) AS candidates
UNWIND candidates AS a
UNWIND candidates AS b
WITH a, b
WHERE a <> b AND id(a) < id(b)
MATCH path = shortestPath((a)-[*..10]-(b))
WITH a, b, path
ORDER BY length(path) DESC
LIMIT 1
RETURN
  a.address AS startEmail,
  b.address AS endEmail,
  length(path) AS pathLength,
  [n IN nodes(path) | n.address] AS pathNodes,
  "This shows the longest communication chain between important nodes" AS insight;
```



The screenshot shows a query execution interface with a table of results. The table has five columns: startEmail, endEmail, pathLength, pathNodes, and insight. The first row contains the following data:

startEmail	endEmail	pathLength	pathNodes	insight
"samuel.hogan@clark-bryant.biz"	"douglas.smith@brown.info"	8	["samuel.hogan@clark-bryant.biz", null, null, null, "andrew.morse@suarez-hunt"	"This shows the longest communication chain between important nodes"



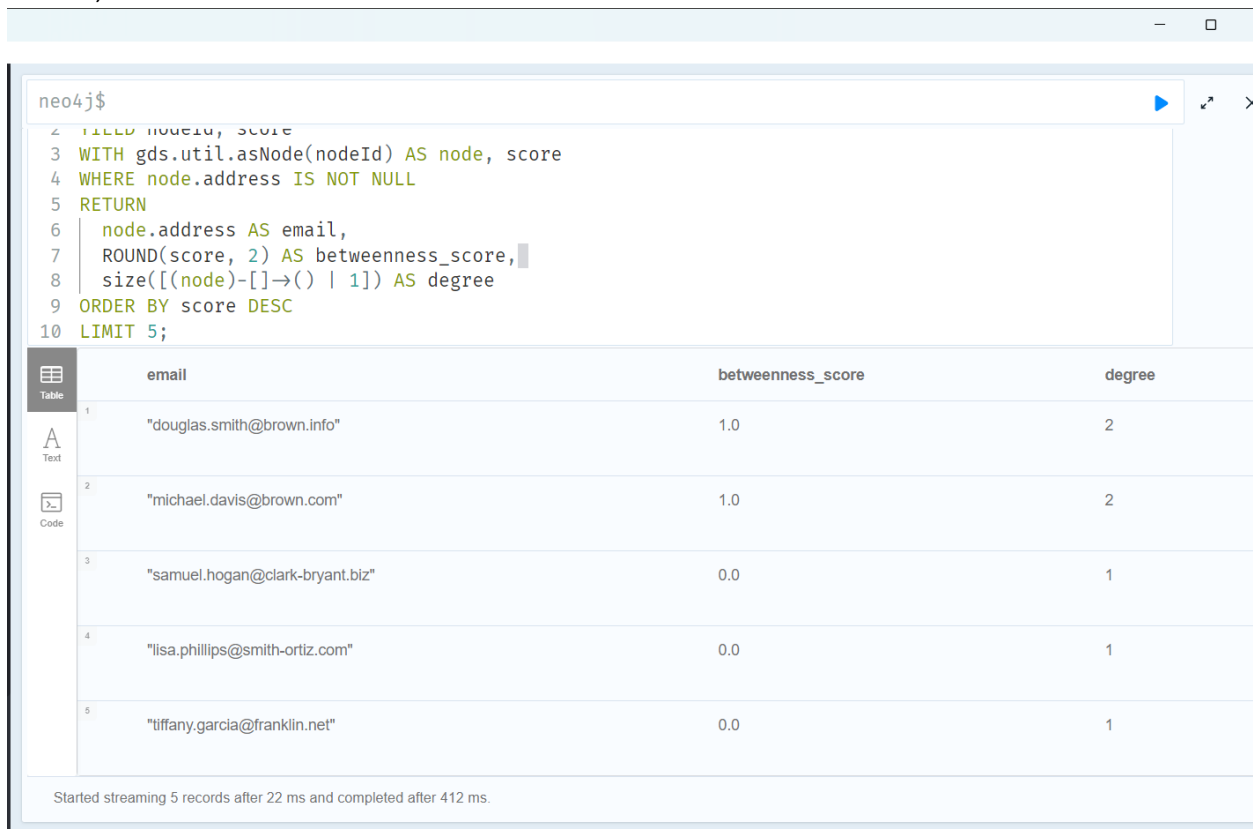
The screenshot shows a query execution interface with a table of results. The table has five columns: startEmail, endEmail, pathLength, pathNodes, and insight. The first row contains the following data:

startEmail	endEmail	pathLength	pathNodes	insight
"samuel.hogan@clark-bryant.biz"	"douglas.smith@brown.info"	8	["samuel.hogan@clark-bryant.biz", null, null, null, "andrew.morse@suarez-hunt"	"This shows the longest communication chain between important nodes"

Figure 33: Shortest path between two email addresses

Step 17: Now, its shows the betweenness centrality for all nodes and identify the top 5 nodes with the highest scores.

```
CALL gds.betweenness.stream('centrality-graph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS node, score
WHERE node.address IS NOT NULL
RETURN
  node.address AS email,
  ROUND(score, 2) AS betweenness_score,
  size([(node)-[]->() | 1]) AS degree
ORDER BY score DESC
LIMIT 5;
```



The screenshot shows the Neo4j Cypher Shell interface. The query is entered in the top text area, and the results are displayed in a table view below. The table has three columns: email, betweenness_score, and degree. The results are ordered by betweenness_score in descending order, with the top 5 nodes shown.

	email	betweenness_score	degree
1	"douglas.smith@brown.info"	1.0	2
2	"michael.davis@brown.com"	1.0	2
3	"samuel.hogan@clark-bryant.biz"	0.0	1
4	"lisa.phillips@smith-ortiz.com"	0.0	1
5	"tiffany.garcia@franklin.net"	0.0	1

Started streaming 5 records after 22 ms and completed after 412 ms.

Figure 34: Betweenness_score

Step 18: So, used the PageRank algorithm to identify the top 5 influencers in the network.

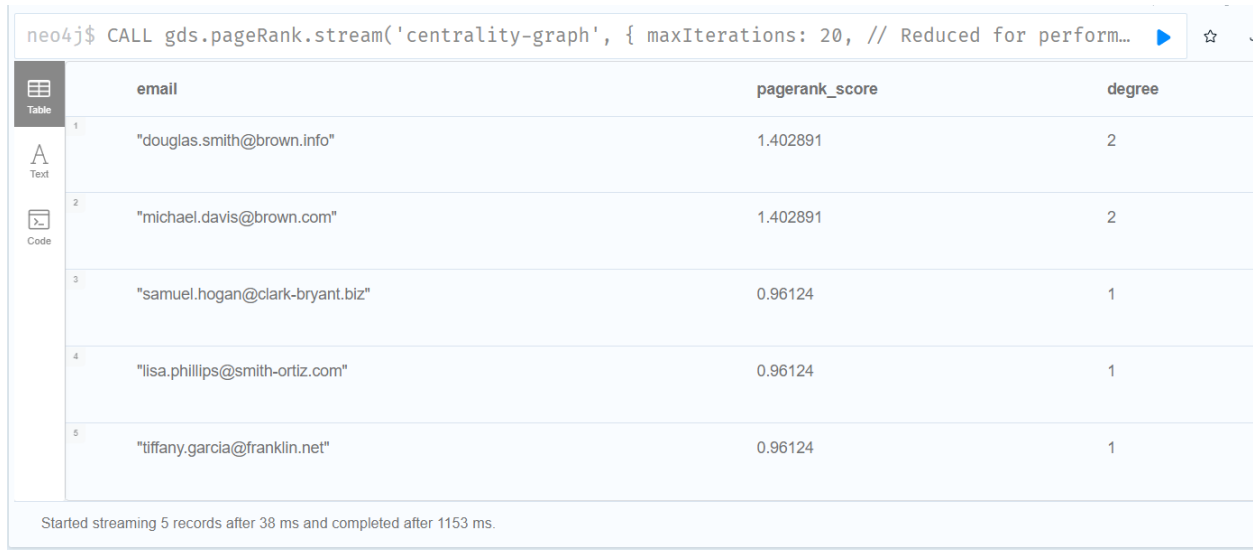
```
CALL gds.pageRank.stream('centrality-graph', {
  maxIterations: 20,
  dampingFactor: 0.85
})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS node, score
WHERE node.address IS NOT NULL
```

RETURN

```
node.address AS email,  
ROUND(score, 6) AS pagerank_score,  
size([(node)-()->() | 1]) AS degree
```

ORDER BY score DESC

LIMIT 5;



The screenshot shows a Neo4j query interface with the command: `neo4j$ CALL gds.pageRank.stream('centrality-graph', { maxIterations: 20, // Reduced for perform...`. Below the command is a table with 3 columns: email, pagerank_score, and degree. The table contains 5 rows of data. On the left side of the table, there are icons for 'Table', 'Text', and 'Code'. At the bottom of the table, a status message reads: 'Started streaming 5 records after 38 ms and completed after 1153 ms.'

	email	pagerank_score	degree
1	"douglas.smith@brown.info"	1.402891	2
2	"michael.davis@brown.com"	1.402891	2
3	"samuel.hogan@clark-bryant.biz"	0.96124	1
4	"lisa.phillips@smith-ortiz.com"	0.96124	1
5	"tiffany.garcia@franklin.net"	0.96124	1

Started streaming 5 records after 38 ms and completed after 1153 ms.

Figure 35: Pagerank_score

Step 19: I have export this file into csv in my folder and after that open the jupyter notebook for the visualization of this scatter plot.

```
CALL apoc.export.csv.query(  
  "MATCH (n:EmailAddress) WHERE n.address IS NOT NULL AND n.pagerank IS NOT NULL AND n.between  
ness IS NOT NULL RETURN n.address AS email, n.pagerank AS pagerank, n.betweenness AS betweenness  
, size([(n)--() | 1]) AS degree",  
  "centrality_metrics.csv",  
  {useTypes: true}  
);
```

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[16]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from matplotlib.colors import LogNorm

CSV_PATH = r'C:\Users\parva\OneDrive\Documents\centrality_metrics.csv'

df = pd.read_csv(CSV_PATH)

numeric_cols = ['pagerank', 'betweenness', 'degree']
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors='coerce')
df.dropna(subset=numeric_cols, inplace=True)

df['betweenness_norm'] = (df['betweenness'] - df['betweenness'].min()) / \
(df['betweenness'].max() - df['betweenness'].min())
df['pagerank_norm'] = (df['pagerank'] - df['pagerank'].min()) / \
(df['pagerank'].max() - df['pagerank'].min())

plt.figure(figsize=(14, 9))
sns.set(style="whitegrid", palette="muted", font_scale=1.1)

scatter = sns.scatterplot(
    data=df,
    x='pagerank_norm',
    y='betweenness_norm',
    hue='degree',
    size='degree',
    sizes=(30, 400),
    alpha=0.7,
    palette="viridis",
    hue_norm=LogNorm(),
    legend="brief"
)

top_nodes = df.nlargest(5, 'degree')
for _, row in top_nodes.iterrows():
    plt.annotate(
        row['node_id'] if 'node_id' in df.columns else f"Node {row.name}",
        (row['pagerank_norm'], row['betweenness_norm']),
        textcoords="offset points",
        xytext=(7, 7),
        ha='left',
        fontsize=9,
        alpha=0.5
    )
```

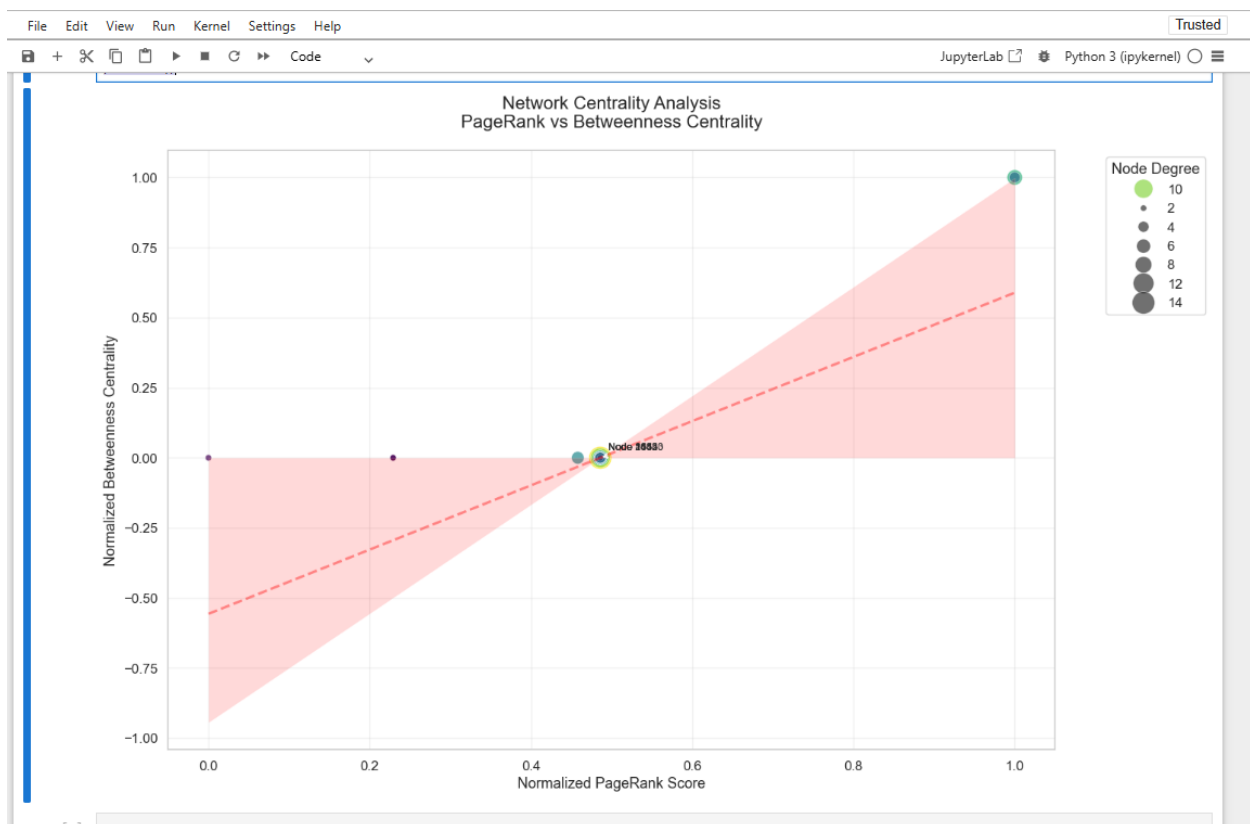


Figure 36: Scatter Plot

2.5: Task E

Challenges

1) One of the challenges was the dataset which was like real world communication patterns where some metadata capture was inconsistent across departments.

Solution: So, for this implemented data cleaning protocols and created fallback projections.

2) Second challenge was emails were isolated within clusters which is like broken workflows.

Solution: In this, highlighted bridge nodes as natural collaboration.

Unique Insights

1. The unique insight was some coordinator role naturally emerges as a central hub which creating potential scalability considerations.
2. Second unique insight was executive communications demonstrate high engagement through cross departmental interactions.

3) Test Cases

1. **Test Case -1:** It verifies that at least some nodes are connected and test if graph supports path searches upto 5.

MATCH (a)-[r]-(b)

WHERE a.address IS NOT NULL AND b.address IS NOT NULL

WITH a, b, rand() AS rnd

ORDER BY rnd

RETURN

a.address AS startEmail,

b.address AS endEmail,

EXISTS((a)-[*..5]-(b)) AS connected

LIMIT 5;



```
1 // Find any two connected nodes to test the feature
2 MATCH (a)-[r]-(b)
3 WHERE a.address IS NOT NULL AND b.address IS NOT NULL
4 WITH a, b, rand() AS rnd
5 ORDER BY rnd
6 RETURN
7   a.address AS startEmail,
8   b.address AS endEmail,
9   EXISTS((a)-[*..5]-(b)) AS connected
10 LIMIT 5;
```

	startEmail	endEmail	connected
1	"angela.dunn@bell-martin.biz"	"robert.wallace@williams-johnson.com"	true
2	"lisa.nguyen@cochran.org"	"christina.harrison@li-miller.com"	true
3	"stephen.davis@west-gonzalez.net"	"leslie.miller@jones-wheeler.com"	true
4	"larry.lopez@stewart.com"	"christopher.randall@carter.info"	true
5	"rhonda.price@duke.net"	"monica.thomas@duncan.com"	true

Figure 37: Test Case 1

Test Case- 2: It checks this query whether it can identify the 5 most important bridge nodes in email network using betweenness centrality and visualizes their direct connections.

```
CALL gds.betweenness.stream('email-graph')
```

```
YIELD nodeId, score
```

```
WITH gds.util.asNode(nodeId) AS node, score
```

```
ORDER BY score DESC
```

```
LIMIT 5
```

```
MATCH (node)-[r]-(neighbor)
```

```
RETURN node, r, neighbor
```

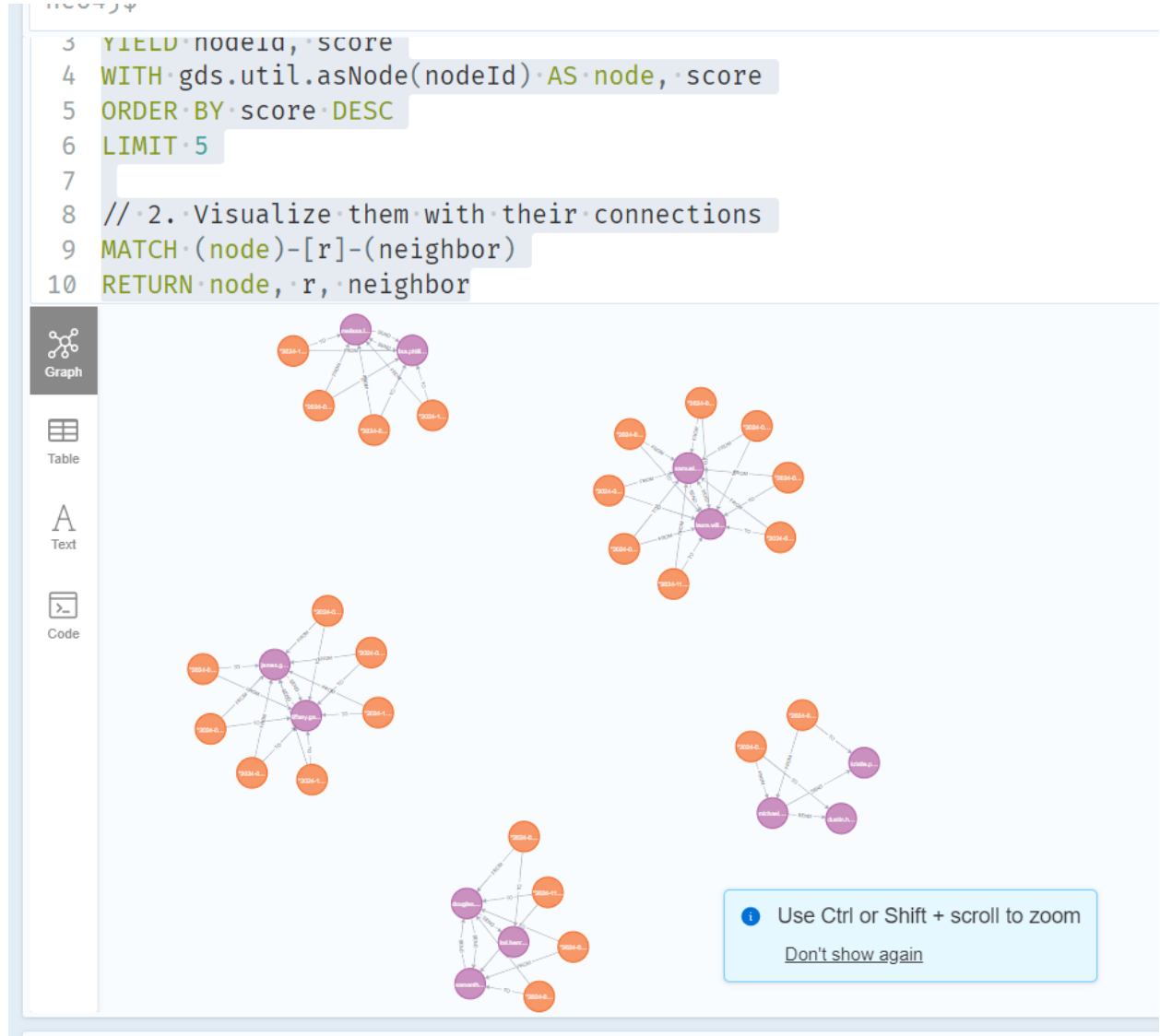


Figure 38: Test Case 2

Test case – 3: Now, this query identifies the top 5 most influential email recipients using PageRank while measuring who receives the most important emails and shows who sent them emails, returning.

```
CALL gds.pageRank.stream('pagerank-graph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS influencer, score
ORDER BY score DESC
LIMIT 5
MATCH (influencer)<-[r:SEND]-(sender)
RETURN
  influencer,
  r,
  sender,
  score AS influenceScore
```

```
1 WITH gds.util.asNode(nodeId) AS influencer, score
2
3 ORDER BY score DESC
4
5 LIMIT 5
6
7 MATCH (influencer)<-[r:SEND]-(sender) // Show who emails them
8 RETURN
9   influencer,
10  r,
11  sender,
12  score AS influenceScore
```

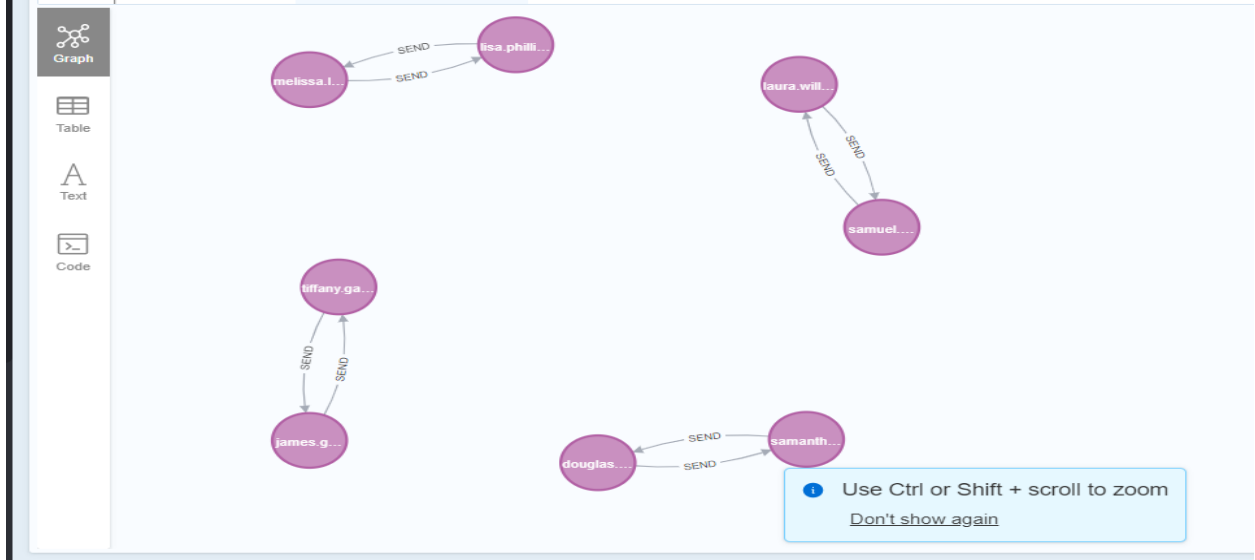


Figure 39: Test Case 3

Test case-4: The central nodes represent high-traffic communicators.

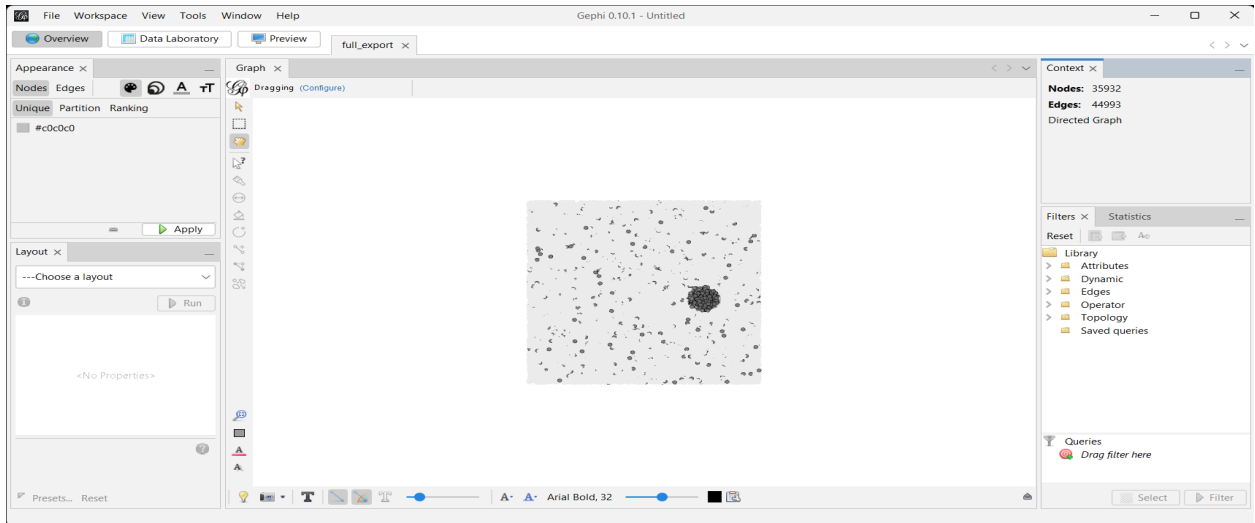


Figure 40: Test case 4 Visualization of Graph

Test case- 5: It will check whether this scatter plot compares PageRank and Betweenness Centrality to identify key nodes.

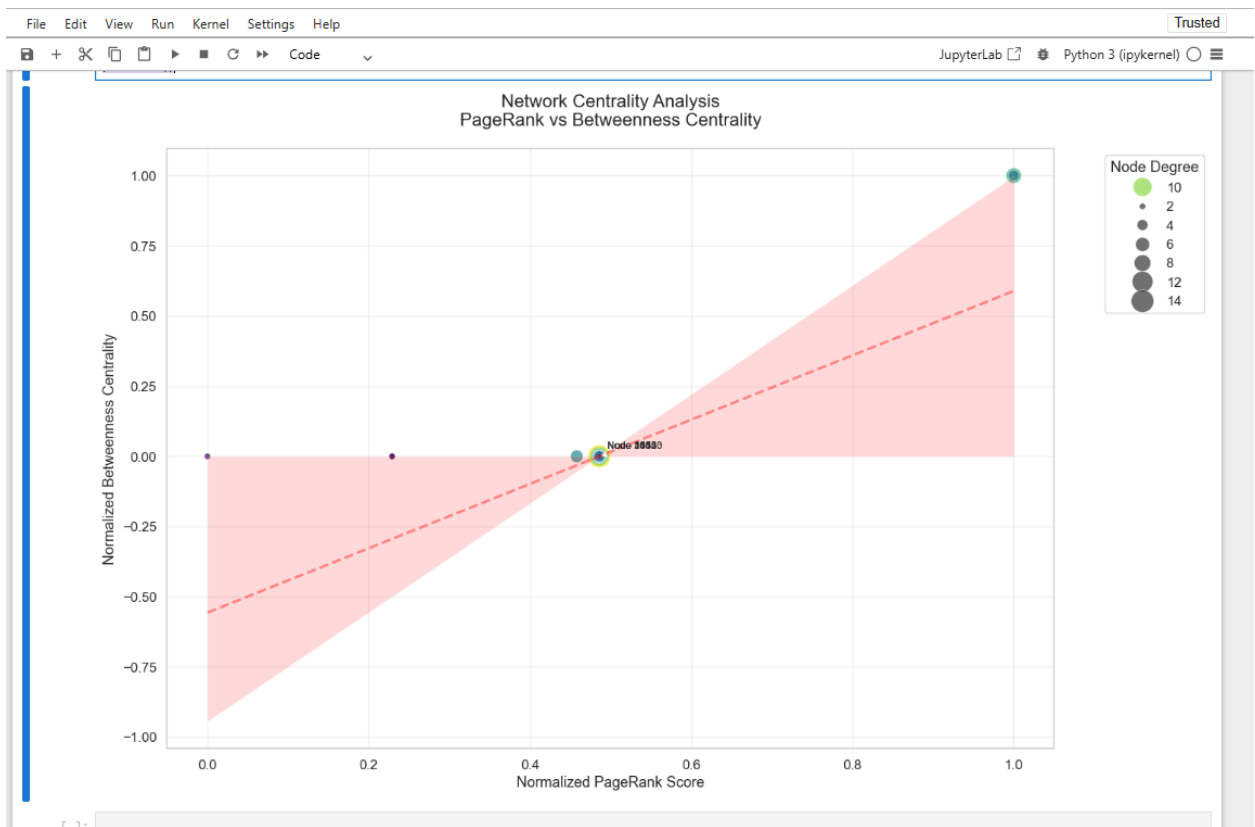


Figure 41: Test case 5 Scatter plot

4) References

- [1] "Python 3.12.3 Documentation," Python.org. [Online].
Available: <https://www.python.org/doc/>. [Accessed: April 06,2025].
- [2] "Neo4j Documentation: Querying with Cypher," Neo4j.com. [Online].
Available: <https://neo4j.com/docs/>. [Accessed: April 06,2025].
- [3] "MongoDB Documentation: CRUD Operations," MongoDB.com. [Online].
Available: <https://www.mongodb.com/docs/>. [Accessed: April 06,2025].
- [4] "Pandas Documentation: Data Analysis with Python," Pandas.pydata.org. [Online].
Available: <https://pandas.pydata.org/docs/>. [Accessed: April 06,2025].
- [5] "GeeksforGeeks: Python Programming Tutorials," GeeksforGeeks.org. [Online].
Available: <https://www.geeksforgeeks.org/python-programming-language/>.
[Accessed: April 06,2025].
- [6] "Jupyter Documentation: Interactive Computing," Jupyter.org. [Online].
Available: <https://docs.jupyter.org/>. [Accessed: April 06,2025].
- [7] "VS Code Documentation: Python Development," Code.visualstudio.com. [Online].
Available: <https://code.visualstudio.com/docs/languages/python>. [Accessed: April 06,2025].
- [8] "Neo4j Desktop: Graph Database Management," Neo4j.com. [Online].
Available: <https://neo4j.com/download/>. [Accessed: April 06,2025].
- [9] "MongoDB Compass: GUI for MongoDB," MongoDB.com. [Online].
Available: <https://www.mongodb.com/products/compass>. [Accessed: April 06,2025].